

Lección 2 — Passive Reconnaissance

1. Introducción General de la Lección

Passive Reconnaissance constituye el primer paso fundamental en cualquier operación de ciberseguridad profesional. A diferencia de técnicas activas que interactúan directamente con sistemas objetivo, el reconnaissance pasivo extrae información valiosa de fuentes públicas sin generar trazas de actividad en los sistemas objetivo.

La información recopilada durante esta fase incluye datos de **DNS enumeration**, **SSL/TLS certificate analysis**, **geolocalización**, **threat intelligence**, y **attack surface mapping**. Estos datos permiten construir un mapa completo de la infraestructura objetivo, identificar servicios expuestos, detectar vulnerabilidades conocidas y establecer vectores de ataque potenciales.

En esta lección se abordan técnicas modernas de passive reconnaissance utilizando herramientas de línea de comandos tradicionales (whois, dig, nslookup) integradas con Python, junto con APIs públicas especializadas como Shodan, Censys, IPInfo y APIVoid. El enfoque es automatizar y escalar estas operaciones mediante scripts Python, permitiendo análisis exhaustivos de infraestructuras complejas.

2. Objetivos

Los objetivos de aprendizaje para esta lección son:

- Dominar técnicas de **DNS enumeration** utilizando herramientas tradicionales (whois, dig, nslookup) y sus equivalentes en Python.
- Comprender cómo automatizar búsquedas avanzadas mediante **Google Dorks** y **Google Hacking**.
- Integrar APIs especializadas de threat intelligence (Shodan, Censys, IPInfo, APIVoid) en flujos de trabajo automatizados.
- Realizar análisis de certificados SSL/TLS para identificar subdominios y superficies de ataque.
- Extraer, parsear e interpretar información de múltiples fuentes para construir perfiles de infraestructura.
- Comprender las implicancias legales y éticas del reconnaissance pasivo.

3. Conceptos Técnicos Fundamentales

Passive Reconnaissance

El reconnaissance pasivo se define como la recopilación de información sobre sistemas objetivo sin interacción directa con ellos. La información proviene de fuentes públicas indexadas por terceros, lo que implica que no genera alertas en sistemas de detección de intrusiones (IDS/IPS) ni registros de acceso directo.

DNS Enumeration

La enumeración de DNS es el proceso de consultar registros DNS públicos para mapear la infraestructura de un dominio. Los registros DNS comúnmente consultados incluyen:

- ****A****: Mapea un nombre de dominio a una dirección IPv4.
- ****AAAA****: Mapea un nombre de dominio a una dirección IPv6.
- ****NS****: Identifica los servidores DNS autorizados para un dominio.
- ****MX****: Especifica los servidores de correo electrónico asociados.
- ****CNAME****: Alias de nombre canónico que apunta a otro dominio.
- ****TXT****: Registros de texto utilizados para verificación de dominio, políticas SPF/DKIM/DMARC.
- ****SOA****: Información de autoridad de inicio de zona.
- ****PTR****: Reverse DNS lookup para resolver IPs a nombres de dominio.

SSL/TLS Certificate Analysis

Los certificados SSL/TLS públicos son registrados en bases de datos públicas (como Certificate Transparency logs). El análisis de estos certificados permite identificar:

- Subdominios asociados a un dominio principal.
- Historial de certificados emitidos.
- Autoridades certificadoras utilizadas.
- Fechas de emisión y expiración.
- Información de contacto del propietario.

Threat Intelligence y Reputation Analysis

Los servicios de threat intelligence mantienen bases de datos de IPs y dominios asociados a actividad maliciosa. Consultar estas bases permite identificar:

- IPs blacklistadas por spam, malware o ataques.
- Dominios registrados recientemente (indicador de infraestructura maliciosa).
- Servicios sospechosos o configuraciones inseguras.
- Uso de VPN, proxies o redes Tor.

Attack Surface Management

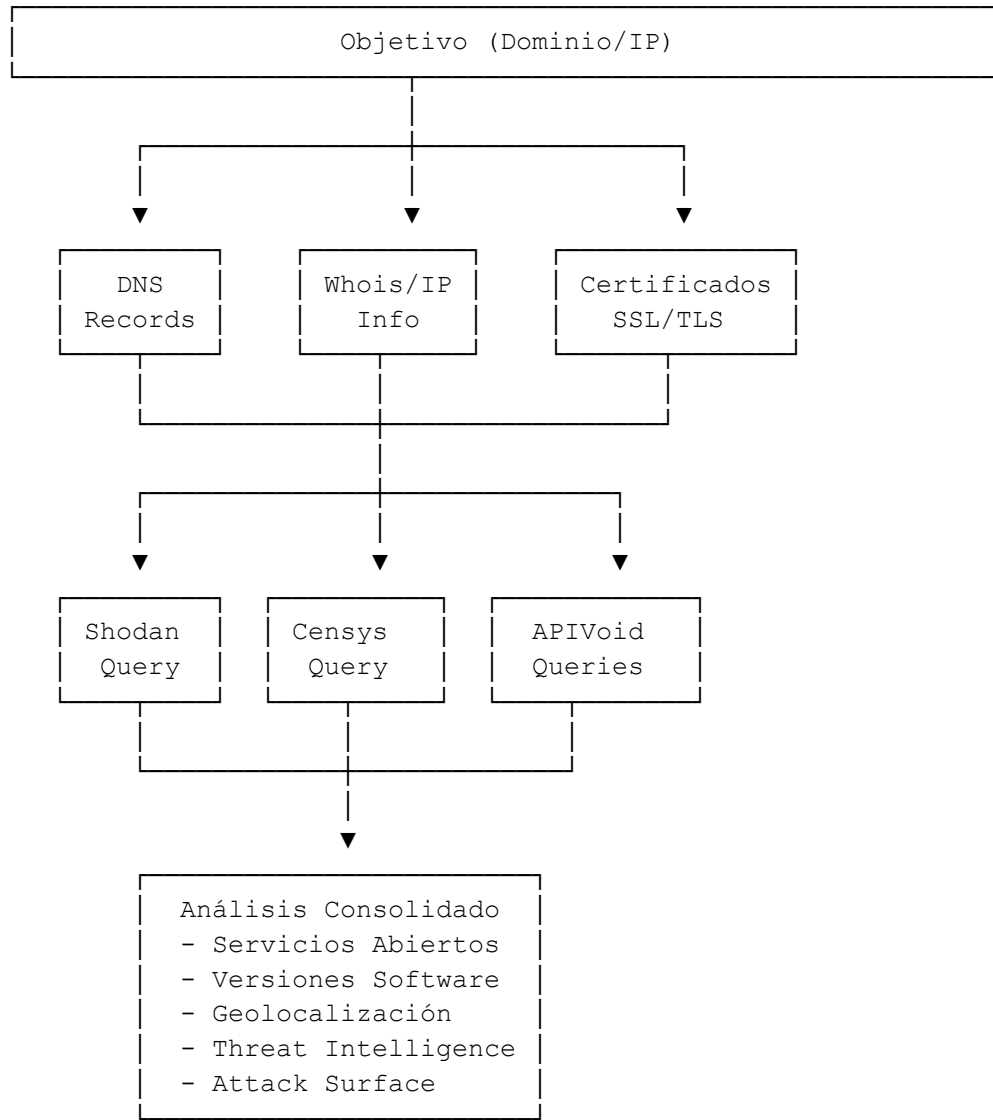
El mapeo de la superficie de ataque (attack surface) implica identificar todos los puntos de entrada potenciales en una infraestructura. Esto incluye servidores web, APIs expuestas, servicios en la nube, dispositivos IoT conectados y configuraciones mal aseguradas.

Open Source Intelligence (OSINT)

OSINT es la disciplina de recopilar, analizar e interpretar información disponible públicamente. Las técnicas de passive reconnaissance son un subconjunto de OSINT enfocado en infraestructura técnica.

4. Arquitectura o Flujo Técnico

El flujo típico de una operación de passive reconnaissance sigue esta arquitectura:



5. Herramientas Utilizadas

Herramientas de Línea de Comandos

Las siguientes herramientas de línea de comandos son fundamentales para passive reconnaissance:

- **whois**: Consulta información de registro de dominios e IPs.
- **dig**: Herramienta DNS avanzada para consultar registros DNS específicos.
- **nslookup**: Utilidad DNS para resolución de nombres e IP addresses.

Librerías Python Requeridas

```
pip install python-whois
pip install pydig
pip install nslookup
pip install google-search-results
```

```
pip install pagodo
pip install shodan
pip install censys
pip install ipinfo
pip install requests
pip install pycrtsh
```

APIs Especializadas

Las siguientes APIs especializadas se utilizan en esta lección:

- ****Shodan****: Motor de búsqueda para dispositivos conectados a Internet. Requiere API key (disponible en <https://www.shodan.io/>).
- ****Censys****: Plataforma de discovery y attack surface management. Requiere API credentials (disponible en <https://censys.io/>).
- ****IPInfo****: Servicio de geolocalización e información de ASN. Requiere access token (disponible en <https://ipinfo.io/>).
- ****APIVoid****: Suite de APIs para threat intelligence. Requiere API key (disponible en <https://www.apivoid.com/>).
- ****crt.sh****: Base de datos pública de certificados SSL/TLS. No requiere autenticación (disponible en <https://crt.sh/>).

6. Scripts de la Lección

6.1 Script: 01_nslookup.py

Introducción del Script

Este script realiza una consulta DNS básica utilizando la librería estándar `socket` de Python. Funciona como equivalente automatizado del comando `nslookup` de línea de comandos, permitiendo resolver nombres de dominio a direcciones IP.

****Propósito****: Validar resolución DNS y mapear infraestructura de dominio.

****Tipo****: Herramienta de reconnaissance pasivo, automatización de DNS enumeration.

Requisitos

- ****Librerías Python****: `socket` (librería estándar, no requiere pip install)
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno

Dependencias Externas

- Acceso a Internet para consultar servidores DNS públicos.
- No requiere API keys ni autenticación.

Explicación Línea por Línea

```
import socket
```

Importa la librería estándar `socket` que proporciona acceso a funciones de red a nivel de sistema operativo.

```
class NetworkUtilites:
    def nslookup(self, domain):
```

Define una clase `NetworkUtilites` (nota: contiene typo en el nombre original del código) con un método `nslookup()` que acepta un dominio como parámetro.

```
        nslookup_result_ip = socket.gethostbyname(domain)
```

Utiliza `socket.gethostbyname()` para resolver el nombre de dominio a una dirección IPv4. Esta función consulta el servidor DNS configurado en el sistema operativo.

```
        print(nslookup_result_ip)
```

Imprime la dirección IP resuelta.

```
def test() -> None:
    network_utilities = NetworkUtilites()
    network_utilities.nslookup("google.com")
test()
```

Bloque de ejecución que instancia la clase y ejecuta una consulta de ejemplo sobre google.com. La función `test()` se ejecuta inmediatamente al cargar el módulo.

Cómo Ejecutar el Script

```
python 01_nslookup.py
```

****Ejemplo con dominio personalizado:****

```
python -c "
import socket
ip = socket.gethostbyname('example.com')
print(ip)
"
```

Resultado Esperado

El script imprime la dirección IP resuelta. Ejemplo:

```
142.250.185.46
```

Riesgos y Consideraciones Éticas

- ****Legalidad****: Las consultas DNS son operaciones pasivas y completamente legales.
- ****Detección****: No generan alertas en sistemas de seguridad.
- ****Impacto****: Ninguno sobre el sistema objetivo.

Troubleshooting

- ****Error "Name or service not known"****: El dominio no existe o está mal escrito.
- ****Timeout en consultas****: Verificar conectividad a Internet.

- ****Resolución incorrecta****: El DNS local puede estar configurado incorrectamente. Usar ``socket.gethostbyname_ex()`` para más detalles.

6.2 Script: `whois_info.py`

Introducción del Script

Este script consulta información WHOIS pública de dominios e IPs. WHOIS es un protocolo que permite acceder a bases de datos de registro de dominios, proporcionando información sobre propietarios, fechas de registro, servidores DNS y datos de contacto.

****Propósito****: Extraer información de registro de dominios para validar autenticidad y realizar análisis histórico.

****Tipo****: Herramienta de reconnaissance pasivo, análisis de registro de dominio.

Requisitos

- ****Librerías Python****: ``whois``
- ****Instalación****: ``pip install python-whois``
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno

Dependencias Externas

- Acceso a Internet para consultar servidores WHOIS públicos.
- No requiere API keys ni autenticación.

Explicación Línea por Línea

```
import whois as ws
```

Importa la librería ``whois`` con alias ``ws`` para consultar información WHOIS. El nombre del paquete pip es ``python-whois``.

```
class WhoisInfo:
    def print_whois_info(self, host):
```

Define una clase ``WhoisInfo`` con método ``print_whois_info()`` que acepta un host (dominio o IP).

```
    whois_info = ws.whois(host)
```

Ejecuta una consulta WHOIS contra el host especificado. La librería contacta automáticamente al servidor WHOIS apropiado basado en la extensión del dominio o rango de IP.

```
    print(whois_info)
```

Imprime el objeto WHOIS completo, que contiene atributos como:

- `registrar`: Entidad que registró el dominio
- `creation\_date`: Fecha de creación del dominio
- `expiration\_date`: Fecha de expiración
- `name\_servers`: Servidores DNS autorizados
- `org`: Organización propietaria
- `country`: País de registro

```
if __name__ == "__main__":  
    whois_info = WhoisInfo()  
    whois_info.print_whois_info("google.com")
```

Bloque de ejecución que instancia la clase y consulta información WHOIS de google.com.

Cómo Ejecutar el Script

```
python whois_info.py
```

****Ejemplo con dominio personalizado:****

```
python -c "  
import whois  
info = whois.whois('example.com')  
print(info)  
"
```

****Ejemplo con dirección IP:****

```
python -c "  
import whois  
info = whois.whois('8.8.8.8')  
print(info)  
"
```

Resultado Esperado

El script imprime información estructurada del registro WHOIS, incluyendo:

- Registrador del dominio
- Fechas de creación y expiración
- Servidores DNS autorizados
- Información de contacto (nombre, organización, país)
- Historial de actualizaciones

Ejemplo de salida parcial:

```
Domain Name: GOOGLE.COM  
Registry Domain ID: 2138514_DOMAIN_COM-VRSN  
Registrar WHOIS Server: whois.markmonitor.com  
Registrar URL: http://www.markmonitor.com  
Updated Date: 2019-09-09T15:39:04Z  
Creation Date: 1997-09-15T04:00:00Z
```

Registry Expiry Date: 2028-09-14T04:00:00Z
Registrar: MarkMonitor Inc.

Riesgos y Consideraciones Éticas

- ****Legalidad****: Las consultas WHOIS son operaciones pasivas y legales.
- ****Privacidad****: Algunos dominios utilizan servicios de privacidad WHOIS que ocultan información de contacto.
- ****Rate Limiting****: Algunos servidores WHOIS pueden limitar consultas frecuentes.

Troubleshooting

- ****Error "ModuleNotFoundError: No module named 'whois'"****: Ejecutar `pip install python-whois`.
- ****Timeout en consultas****: El servidor WHOIS puede estar lento. Reintentar después de unos segundos.
- ****Información incompleta****: Algunos registradores no publican toda la información WHOIS.

6.3 Script: dig_info.py

Introducción del Script

Este script utiliza la librería `pydig` para ejecutar consultas DNS avanzadas equivalentes al comando `dig` de línea de comandos. Permite consultar tipos específicos de registros DNS (A, MX, NS, TXT, etc.) con control fino sobre los parámetros de consulta.

****Propósito****: Realizar DNS enumeration detallada y mapear infraestructura de dominio.

****Tipo****: Herramienta de reconnaissance pasivo, DNS enumeration avanzada.

Requisitos

- ****Librerías Python****: `pydig`
- ****Instalación****: `pip install pydig`
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno

Dependencias Externas

- Acceso a Internet para consultar servidores DNS.
- No requiere API keys ni autenticación.

Explicación Línea por Línea

```
import pydig as pd
```

Importa la librería `pydig` con alias `pd` que proporciona interfaz Python para el comando `dig`.

```
class DigInfo:
    def print_dig_info(self, host, type):
```


Define una clase `DigInfo` con método `print\_dig\_info()` que acepta un dominio y tipo de registro.

```
dig_info = pd.query(host, type)
```

Ejecuta una consulta DNS usando `pd.query()`, especificando el dominio y tipo de registro a consultar.

```
print(dig_info)
```

Imprime los resultados de la consulta, que pueden incluir múltiples registros del tipo especificado.

```
if __name__ == "__main__":
    dig_info = DigInfo()
    dig_info.print_dig_info('google.com', 'A')
    dig_info.print_dig_info('google.com', 'NS')
```

Bloque de ejecución que consulta registros A (IPv4) y NS (Name Servers) de google.com.

Cómo Ejecutar el Script

```
python dig_info.py
```

****Ejemplo consultando diferentes tipos de registros:****

```
python -c "
import pydig as pd
# Registros A (IPv4)
print('A Records:', pd.query('example.com', 'A'))
# Registros MX (Mail Exchange)
print('MX Records:', pd.query('example.com', 'MX'))
# Registros NS (Name Servers)
print('NS Records:', pd.query('example.com', 'NS'))
# Registros TXT
print('TXT Records:', pd.query('example.com', 'TXT'))
"
```

Resultado Esperado

El script retorna los registros DNS del tipo especificado. Ejemplos:

****Registros A:****

```
['142.250.185.46']
```

****Registros MX:****

```
['10 smtp.google.com.']
```

****Registros NS:****

```
['ns1.google.com.', 'ns2.google.com.', 'ns3.google.com.',  
'ns4.google.com.']
```

Riesgos y Consideraciones Éticas

- ****Legalidad****: Las consultas DNS son operaciones pasivas y legales.
- ****Detección****: No generan alertas en sistemas de seguridad.

Troubleshooting

- ****Error "ModuleNotFoundError: No module named 'pydig'"****: Ejecutar `pip install pydig`.
- ****Registros vacíos****: El dominio puede no tener registros del tipo especificado.

6.4 Script: nslookup_info.py

Introducción del Script

Este script utiliza la librería `nslookup` para realizar consultas DNS. Proporciona una interfaz más limpia para resolver dominios a direcciones IP utilizando servidores DNS específicos.

****Propósito****: Resolución de DNS con control de servidor DNS y análisis de respuesta completa.

****Tipo****: Herramienta de reconnaissance pasivo, DNS resolution avanzada.

Requisitos

- ****Librerías Python****: `nslookup`
- ****Instalación****: `pip install nslookup`
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno

Dependencias Externas

- Acceso a Internet para consultar DNS públicos.
- No requiere API keys ni autenticación.

Explicación Línea por Línea

```
from nslookup import Nslookup
```

Importa la clase `Nslookup` de la librería homónima.

```
class NsLookup:  
    def nslookup(self, domain):
```

Define clase con método que acepta un dominio.

```
dns_query = Nslookup(dns_servers=["8.8.8.8"])
```

Instancia un objeto `Nslookup` especificando que debe usar Google DNS (8.8.8.8) como servidor DNS. Esto asegura consistencia en las consultas independientemente de la configuración DNS local.

```
dns_record = dns_query.dns_lookup(domain)
```

Ejecuta la consulta DNS contra el dominio especificado, retornando un objeto con los resultados.

```
print(dns_record.response_full, dns_record.answer)
```

Imprime tanto la respuesta completa del servidor DNS como las respuestas parseadas (direcciones IP resueltas).

```
if __name__ == "__main__":  
    ns_lookup = NsLookup()  
    ns_lookup.nslookup("google.com")
```

Bloque de ejecución que instancia la clase y ejecuta una consulta sobre google.com.

Cómo Ejecutar el Script

```
python nslookup_info.py
```

Resultado Esperado

Salida con resolución DNS completa incluyendo respuesta del servidor y direcciones IP.

6.5 Script: google_search.py

Introducción del Script

Este script automatiza búsquedas en Google utilizando la librería `google-search-results`. Permite ejecutar búsquedas programáticas y extraer resultados de manera automatizada.

****Propósito**:** Automatizar búsquedas en Google para descubrir información expuesta.

****Tipo**:** Herramienta de reconnaissance pasivo, OSINT automation.

Requisitos

- ****Librerías Python**:** `google-search-results`
- ****Instalación**:** `pip install google-search-results`
- ****Sistemas operativos compatibles**:** Windows, Linux, macOS
- ****Requisitos de privilegios**:** Ninguno

Dependencias Externas

- Acceso a Internet para consultar Google.
- No requiere API keys (aunque existen versiones con límites de tasa).

Explicación Línea por Línea

```
from googlesearch import search
from sympy import Q
```

Importa la función `search` de la librería `google-search-results`. ****NOTA IMPORTANTE****: El import `from sympy import Q` está presente en el código pero ****NO se utiliza****. Es código muerto que debe ignorarse.

```
class GoogleSearch:
    def perform_google_search(self, host, query_string):
```

Define clase con método que acepta un host y string de búsqueda.

```
        for query in search(host + " " + query_string, stop=5):
            print(query)
```

Ejecuta búsqueda combinando host y query_string, limitando a 5 resultados (`stop=5`), e imprime cada URL resultante.

```
if __name__ == "__main__":
    google_search = GoogleSearch()
    google_search.perform_google_search(
        "site:<<Domain>>", 'inurl:"wp-json/" -wordpress')
```

Ejecuta búsqueda avanzada buscando endpoints de WordPress JSON en un dominio específico.

Cómo Ejecutar el Script

```
python google_search.py
```

****Ejemplo con búsqueda personalizada:****

```
python -c "
from googlesearch import search
for result in search('site:example.com filetype:pdf', stop=5):
    print(result)
"
```

Resultado Esperado

El script imprime URLs encontradas por Google que coinciden con los criterios de búsqueda. Ejemplo:

```
https://example.com/wp-json/wp/v2/users
https://example.com/api/wp-json/
```

Riesgos y Consideraciones Éticas

- ****Rate Limiting****: Google puede bloquear consultas excesivas desde la misma IP.
- ****Términos de Servicio****: El scraping de Google puede violar sus términos de servicio.

- ****Legalidad****: Las búsquedas son pasivas, pero el volumen excesivo puede considerarse ataque de denegación de servicio.

Troubleshooting

- ****Error "ModuleNotFoundError"****: Ejecutar `pip install google-search-results`.
- ****Resultados vacíos****: Google puede estar bloqueando la IP por demasiadas consultas.
- ****Timeout****: Aumentar el intervalo entre búsquedas.

6.6 Script: google_dorks.py

Introducción del Script

Este script automatiza búsquedas avanzadas en Google (Google Dorks) utilizando la herramienta ****Pagodo****. Los Google Dorks son consultas especializadas que aprovechan operadores avanzados de Google para descubrir información sensible, configuraciones inseguras, paneles administrativos expuestos y vulnerabilidades.

****Propósito****: Automatizar búsquedas de información sensible y configuraciones vulnerables.

****Tipo****: Herramienta de reconnaissance pasivo, vulnerability discovery.

Requisitos

- ****Librerías Python****: `pagodo`
- ****Instalación****: `pip install pagodo`
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno

Dependencias Externas

- Acceso a Internet para consultar Google.
- Archivo `dorks.txt` con consultas predefinidas (incluido en el paquete).

Explicación Línea por Línea

```
from pagodo.pagodo import Pagodo
```

Importa la clase `Pagodo` de la librería homónima.

```
class GoogleDorks:
    def process_dorks(self):
```

Define clase con método que procesa Google Dorks.

```
pg = Pagodo(
    google_dorks_file="Chapter 02/dorks.txt",
    domain="<<Domain>>",
    max_search_result_urls_to_return_per_dork=3,
    save_pagodo_results_to_json_file=True,
```

```
        minimum_delay_between_dork_searches_in_seconds=10,  
        maximum_delay_between_dork_searches_in_seconds=15,  
        save_urls_to_file=True,  
        verbosity=4  
    )
```

Instancia objeto `Pagodo` con parámetros de configuración:

- `google\_dorks\_file`: Ruta al archivo con dorks (NOTA: ruta hardcodeada como `"Chapter 02/dorks.txt"`)
- `domain`: Dominio objetivo
- `max\_search\_result\_urls\_to\_return\_per\_dork`: Máximo de URLs por dork
- `minimum\_delay\_between\_dork\_searches\_in\_seconds`: Delay mínimo entre búsquedas
- `maximum\_delay\_between\_dork\_searches\_in\_seconds`: Delay máximo entre búsquedas
- `save\_urls\_to\_file`: Guardar resultados en archivo
- `verbosity`: Nivel de verbosidad

****IMPORTANTE****: La ruta `"Chapter 02/dorks.txt"` es hardcodeada y debe ajustarse a la estructura real del proyecto.

```
pagodo_results_dict = pg.go()
```

Ejecuta el proceso de búsqueda de dorks.

```
for key, value in pagodo_results_dict["dorks"].items():  
    print(f"dork: {key}")  
    for url in value["urls"]:  
        print(url)
```

Itera sobre resultados e imprime dorks y URLs encontradas.

Cómo Ejecutar el Script

```
# Primero, ajustar la ruta del archivo dorks.txt en el código  
python google_dorks.py
```

****Nota sobre la ruta de archivo****: El código contiene `google\_dorks\_file="Chapter 02/dorks.txt"`. Esta ruta debe ajustarse según la estructura real del proyecto. Si el archivo está en el mismo directorio, usar:

```
google_dorks_file="dorks.txt"
```

Resultado Esperado

El script imprime dorks ejecutados y URLs encontradas. Ejemplo:

```
dork: inurl:/wp-content/w3tc/dbcache/  
https://example.com/wp-content/w3tc/dbcache/index.html  
dork: intitle:phpMyAdmin "Welcome to phpMyAdmin"  
https://example.com/admin/phpmyadmin/
```

Riesgos y Consideraciones Éticas

- ****Legalidad****: Los Google Dorks son búsquedas pasivas, pero pueden descubrir información sensible.
- ****Responsabilidad Divulgación****: Si se descubre información sensible, debe reportarse responsablemente.
- ****Rate Limiting****: Google bloquea consultas excesivas.

Troubleshooting

- ****Error "FileNotFoundError" para dorks.txt****: Verificar ruta del archivo y ajustarla en el código.
- ****Resultados limitados****: Google puede estar bloqueando por demasiadas consultas.
- ****Error de configuración de Pagodo****: Verificar que la librería esté correctamente instalada.

6.7 Script: shodan_print_host_details.py

Introducción del Script

Este script consulta la API de Shodan para obtener información completa sobre un host (dirección IP). Shodan es un motor de búsqueda especializado en dispositivos conectados a Internet, indexando puertos abiertos, versiones de servicios, banners y metadatos de configuración.

****Propósito****: Extraer información detallada de infraestructura expuesta en Internet.

****Tipo****: Herramienta de reconnaissance pasivo, threat intelligence.

Requisitos

- ****Librerías Python****: `shodan`
- ****Instalación****: `pip install shodan`
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno
- ****API Key de Shodan****: Requerida (disponible en <https://www.shodan.io/>)

Dependencias Externas

- ****Shodan API****: Requiere API key válida. Disponible en versión gratuita (free tier) con limitaciones de consultas mensuales.
- ****Acceso a Internet****: Para contactar API de Shodan.

Explicación Línea por Línea

```
from shodan import Shodan
import json
```

Importa clase `Shodan` de la librería homónima y módulo `json` para serialización.

```
api = Shodan('<<API Key>>')
```

Instancia cliente de Shodan con API key. ****IMPORTANTE****: Reemplazar `<<API Key>>` con API key válida de Shodan.

```
class ShodanHostDetails:
    def print_host_details(self, ip_address):
```

Define clase con método que acepta dirección IP.

```
        ipinfo = api.host(ip_address)
```

Consulta API de Shodan para obtener información completa del host. Retorna diccionario con datos como:

- `ip\_str`: Dirección IP
- `ports`: Puertos abiertos
- `hostnames`: Nombres de host asociados
- `os`: Sistema operativo detectado
- `org`: Organización propietaria
- `city`, `latitude`, `longitude`: Geolocalización
- `data`: Array con información de cada puerto (banners, versiones, etc.)

```
        print(json.dumps(ipinfo, indent=4))
```

Serializa respuesta a JSON con indentación de 4 espacios para legibilidad.

```
if __name__ == "__main__":
    shodan_info = ShodanHostDetails()
    shodan_info.print_host_details('<<IP Address>>')
```

Bloque de ejecución. ****IMPORTANTE****: Reemplazar `<<IP Address>>` con IP válida.

Cómo Ejecutar el Script

```
# Primero, configurar API key
export SHODAN_API_KEY="your_api_key_here"

# Ejecutar script
python shodan_print_host_details.py
```

****Ejemplo con IP personalizada:****

```
python -c "
from shodan import Shodan
api = Shodan('your_api_key')
host = api.host('8.8.8.8')
import json
print(json.dumps(host, indent=2))
"
```


Resultado Esperado

El script imprime información JSON detallada del host. Ejemplo parcial:

```
{
  "ip_str": "8.8.8.8",
  "ports": [53, 443],
  "hostnames": ["dns.google"],
  "org": "Google LLC",
  "os": "Linux",
  "city": "Mountain View",
  "country_name": "United States",
  "latitude": 37.386,
  "longitude": -122.084
}
```

Riesgos y Consideraciones Éticas

- ****Legalidad****: Consultar Shodan es pasivo y legal.
- ****Información Sensible****: Shodan puede exponer servicios mal configurados. Usar responsablemente.
- ****Rate Limiting****: API gratuita tiene límites de consultas mensuales.

Troubleshooting

- ****Error "Invalid API key"****: Verificar que API key sea válida.
- ****Error "Quota exceeded"****: Límite mensual de consultas alcanzado. Esperar al mes siguiente o actualizar plan.
- ****Error "Host not found"****: IP no indexada en Shodan.

6.8 Script: shodan_parse_host_details.py

Introducción del Script

Este script consulta Shodan y parsea atributos específicos de la respuesta (ciudad, coordenadas geográficas, país). Permite extraer información selectiva en lugar de imprimir la respuesta completa.

****Propósito****: Extraer atributos específicos de información de Shodan.

****Tipo****: Herramienta de reconnaissance pasivo, data parsing.

Requisitos

- ****Librerías Python****: `shodan`
- ****Instalación****: `pip install shodan`
- ****API Key de Shodan****: Requerida

Dependencias Externas

- ****Shodan API****: Requiere API key válida.

Explicación Línea por Línea

```
from shodan import Shodan
import json
```

Importa librerías necesarias.

```
api = Shodan('<<API Key>>')
```

Instancia cliente de Shodan.

```
class ShodanHostDetailsParser:
    def parse_host_details(self, ip_address):
```

Define clase con método de parsing.

```
ipinfo = api.host(ip_address)
```

Obtiene información completa del host.

```
json_data = json.loads(json.dumps(ipinfo))
```

Convierte respuesta a JSON string y luego a objeto Python (técnica para normalizar estructura).

```
print('city: ', json_data.get('city'))
print('latitude: ', json_data.get('latitude'))
print('longitude: ', json_data.get('longitude'))
print('country_name: ', json_data.get('country_name'))
```

Extrae atributos específicos usando método `get()` (retorna `None` si clave no existe).

Cómo Ejecutar el Script

```
python shodan_parse_host_details.py
```

Resultado Esperado

Salida con atributos específicos:

```
city: Mountain View
latitude: 37.386
longitude: -122.084
country_name: United States
```

6.9 Script: censys_print_host_details.py

Introducción del Script

Este script consulta la API de Censys para obtener información sobre un host. Censys es una plataforma de discovery y attack surface management que indexa certificados SSL/TLS, servicios web y dispositivos conectados.

****Propósito**:** Obtener información de infraestructura desde Censys.

****Tipo**:** Herramienta de reconnaissance pasivo, attack surface discovery.

Requisitos

- ****Librerías Python**:** `censys`
- ****Instalación**:** `pip install censys`
- ****Credenciales de Censys**:** User ID y API Secret (disponibles en <https://censys.io/>)

Dependencias Externas

- ****Censys API**:** Requiere credenciales válidas.
- ****Acceso a Internet**:** Para contactar API de Censys.

Explicación Línea por Línea

```
import json
from censys.search import CensysHosts
```

Importa clase `CensysHosts` para consultar información de IPs.

```
censys_host = CensysHosts()
```

Instancia objeto `CensysHosts` a nivel de módulo. La autenticación se realiza automáticamente usando variables de entorno o archivo de configuración.

```
class CensysHostDetails:
    def print_host_details(self, ip_address) -> json:
```

Define clase con método que acepta IP.

```
try:
    ipinfo = censys_host.view(ip_address)
except:
    ipinfo = {}
```

Consulta API de Censys para obtener información del host con manejo de excepciones.

```
print(json.dumps(ipinfo, indent=4))
```

Imprime respuesta en formato JSON.

Cómo Ejecutar el Script

```
python censys_print_host_details.py
```

Resultado Esperado

Información JSON de Censys con servicios, certificados y metadatos.

6.10 Script: censys_parse_host_details.py

Introducción del Script

Similar a `censys\_print\_host\_details.py`, pero parsea atributos específicos de la respuesta, extrayendo información de servicios detectados.

****Propósito**:** Extraer atributos selectivos de información de Censys.

****Tipo**:** Herramienta de reconnaissance pasivo, data parsing.

Requisitos

- ****Librerías Python**:** `censys`
- ****Credenciales de Censys**:** Requeridas

Explicación Línea por Línea

```
import json
from censys.search import CensysHosts
```

Importa librerías.

```
censys_host = CensysHosts()
```

Instancia cliente de Censys.

```
class CensysHostDetailsParser:
    def print_host_details(self, ip_address) -> json:
```

Define clase con método. ****NOTA**:** El nombre del método es `print\_host\_details`, no `parse\_host\_details`, lo que es inconsistente con el nombre de la clase.

```
    try:
        ipinfo = censys_host.view(ip_address)
    except:
        ipinfo = {}
```

Obtiene información con manejo de excepciones.

```
    json_data = json.loads(json.dumps(ipinfo))
```

Normaliza estructura a objeto JSON.

```
        print('[services][0][service_name]: ',
              json_data['services'][0]['service_name'])
        print('[services][0][port]: ',
              json_data['services'][0]['port'])
        print('[services][0][banner]: ',
              json_data['services'][0]['banner'])
```

Imprime atributos específicos del primer servicio detectado.

Cómo Ejecutar el Script

```
python censys_parse_host_details.py
```

Resultado Esperado

Salida con información del primer servicio:

```
[services][0][service_name]: HTTP
[services][0][port]: 80
[services][0][banner]: Apache/2.4.41
```

6.11 Script: ipinfo_ip_details.py

Introducción del Script

Este script consulta la API de IPInfo para obtener información de geolocalización y ASN de una dirección IP. IPInfo proporciona datos como ciudad, país, timezone, ISP y información de ASN (Autonomous System Number).

****Propósito**:** Obtener geolocalización e información de ASN de direcciones IP.

****Tipo**:** Herramienta de reconnaissance pasivo, geolocation intelligence.

Requisitos

- ****Librerías Python**:** `ipinfo`
- ****Instalación**:** `pip install ipinfo`
- ****Access Token de IPInfo**:** Requerido (disponible en <https://ipinfo.io/>)

Dependencias Externas

- ****IPInfo API**:** Requiere access token válido. Disponible versión gratuita con limitaciones.

Explicación Línea por Línea

```
import json
import ipinfo
```

Importa módulos necesarios.

```
ipinfo_accesstoken = '<<Access Token>>'
handler = ipinfo.getHandler(ipinfo_accesstoken)
```

Configura handler de IPInfo con access token. ****IMPORTANTE**:** Reemplazar placeholder con token válido.

```
class IpinfoDetails:
    def get_host_details(self, ip_address):
```

Define clase con método que acepta IP.

```
        try:
            global handler
```

```
        ipinfo_details = handler.getDetails(ip_address)
    except:
        ipinfo_details = {}
```

Consulta API de IPInfo con manejo de excepciones. Si falla, retorna diccionario vacío.

```
print (json.dumps(ipinfo_details.all, indent=4))
```

Imprime todos los detalles en formato JSON.

Cómo Ejecutar el Script

```
python ipinfo_ip_details.py
```

****Ejemplo con IP personalizada:****

```
python -c "
import ipinfo
handler = ipinfo.getHandler('your_token')
details = handler.getDetails('8.8.8.8')
import json
print(json.dumps(details.all, indent=2))
"
```

Resultado Esperado

Información JSON con datos de geolocalización:

```
{
  "ip": "8.8.8.8",
  "city": "Mountain View",
  "region": "California",
  "country": "US",
  "loc": "37.386,-122.084",
  "org": "AS15169 Google LLC",
  "timezone": "America/Los_Angeles"
}
```

Riesgos y Consideraciones Éticas

- ****Legalidad****: Consultar IPInfo es pasivo y legal.
- ****Privacidad****: La geolocalización puede revelar ubicación física de infraestructura.

Troubleshooting

- ****Error "Invalid token"****: Verificar que access token sea válido.
- ****Información incompleta****: Token gratuito tiene limitaciones.

6.12 Script: ipinfo_ip_parser.py

Introducción del Script

Este script consulta IPInfo y parsea atributos específicos (ciudad, país, timezone) en lugar de imprimir la respuesta completa.

****Propósito**:** Extraer atributos selectivos de información de IPInfo.

****Tipo**:** Herramienta de reconnaissance pasivo, data parsing.

Requisitos

- ****Librerías Python**:** `ipinfo`
- ****Access Token de IPInfo**:** Requerido

Explicación Línea por Línea

```
import json
import ipinfo
```

Importa librerías.

```
ipinfo_accesstoken = '<<Access Token>>'
handler = ipinfo.getHandler(ipinfo_accesstoken)
```

Configura handler de IPInfo.

```
class IpinfoDetails:
    def get_host_details(self, ip_address):
```

Define clase con método.

```
    try:
        global handler
        ipinfo_details = handler.getDetails(ip_address)
    except:
        ipinfo_details = {}
```

Obtiene detalles con manejo de excepciones.

```
    json_data = json.loads(json.dumps(ipinfo_details.all,
    indent=4))
```

Normaliza estructura a objeto JSON.

```
    print('city: ', json_data['city'])
    print('country: ', json_data['country'])
    print('timezone: ', json_data['timezone'])
```

Imprime atributos específicos.

Cómo Ejecutar el Script

```
python ipinfo_ip_parser.py
```

Profesor: Patricio Canelo Estay

Resultado Esperado

Salida con atributos específicos:

```
city: Mountain View
country: US
timezone: America/Los_Angeles
```

6.13 Script: apivoid_ip_reputation.py

Introducción del Script

Este script consulta la API de APIVoid para obtener información de reputación de una dirección IP. APIVoid mantiene bases de datos de IPs asociadas a actividad maliciosa, spam, botnets y otros riesgos de seguridad.

****Propósito**:** Evaluar reputación de direcciones IP contra múltiples blacklist engines.

****Tipo**:** Herramienta de reconnaissance pasivo, threat intelligence.

Requisitos

- ****Librerías Python**:** `requests`
- ****Instalación**:** `pip install requests`
- ****API Key de APIVoid**:** Requerida (disponible en <https://www.apivoid.com/>)

Dependencias Externas

- ****APIVoid API**:** Requiere API key válida. Disponible versión gratuita.

Explicación Línea por Línea

```
import requests
import json
```

Importa librerías para HTTP requests y JSON.

```
apivoid_key = "<<API Key>>"
```

Define API key de APIVoid. ****IMPORTANTE****: Reemplazar placeholder.

```
class ApivoidIpReputation:
    def apivoid_query(self, endpoint, ip_address) -> json:
```

Define clase con método genérico que acepta endpoint e IP.

```
        try:
            r =
requests.get(url='https://endpoint.apivoid.com/'+endpoint +
                '/v1/pay-as-you-
go/?key='+apivoid_key+'&ip='+ip_address)
            return json.loads(r.content.decode())
```



```
except:
    return {}
```

Construye URL de API con parámetros, ejecuta GET request y parsea respuesta JSON. En caso de error, retorna diccionario vacío.

```
if __name__ == "__main__":
    apivoid_ip_reputation = ApivoidIpReputation()
    json_data = apivoid_ip_reputation.apivoid_query('iprep',
'<<IP Address>')
    print(json.dumps(json_data, indent=4))
```

Bloque de ejecución que consulta endpoint `iprep` para reputación de IP.

Cómo Ejecutar el Script

```
python apivoid_ip_reputation.py
```

Resultado Esperado

Información JSON con datos de reputación:

```
{
  "data": {
    "report": {
      "ip": "8.8.8.8",
      "is_blacklisted": false,
      "blacklists": {
        "engines": [
          {
            "engine": "Spamhaus",
            "detected": false
          }
        ]
      }
    }
  }
}
```

6.14 Script: apivoid_domain_reputation.py

Introducción del Script

Este script consulta APIVoid para evaluar la reputación de un dominio. Detecta dominios maliciosos, phishing, malware y otras amenazas.

****Propósito**:** Evaluar reputación de dominios contra múltiples threat intelligence engines.

****Tipo**:** Herramienta de reconnaissance pasivo, threat intelligence.

Requisitos

- ****Librerías Python****: `requests`
- ****API Key de APIVoid****: Requerida

Explicación Línea por Línea

```
import requests
import json
```

Importa librerías.

```
apivoid_key = "<<API Key>>"
```

Define API key.

```
class ApivoidDomainReputation:
    def apivoid_query(self, endpoint, host) -> json:
```

Define clase con método genérico que acepta endpoint y host.

```
        try:
            r =
requests.get(url='https://endpoint.apivoid.com/'+endpoint +
                '/v1/pay-as-you-
go/?key='+apivoid_key+'&host='+host)
            return json.loads(r.content.decode())
        except:
            return {}
```

Construye URL de API, ejecuta request y parsea respuesta.

```
if __name__ == "__main__":
    apivoid_domain_reputation = ApivoidDomainReputation()
    json_data =
apivoid_domain_reputation.apivoid_query('domainbl', '<<Host>>')
    print(json.dumps(json_data, indent=4))
```

Bloque de ejecución que consulta endpoint `domainbl` para reputación de dominio.

Cómo Ejecutar el Script

```
python apivoid_domain_reputation.py
```

6.15 Script: apivoid_dns_lookup.py

Introducción del Script

Este script utiliza APIVoid para realizar DNS lookups. Proporciona información de registros DNS con análisis de reputación.

****Propósito****: Realizar DNS enumeration con análisis de reputación.

****Tipo**:** Herramienta de reconnaissance pasivo, DNS enumeration.

Requisitos

- ****Librerías Python**:** `requests`
- ****API Key de APIVoid**:** Requerida

Explicación Línea por Línea

```
import requests
import json
```

Importa librerías.

```
apivoid_key = "<<API Key>>"
```

Define API key.

```
class ApivoidDnsLookup:
    def apivoid_query(self, endpoint, host, action) -> json:
```

Define clase con método que acepta endpoint, host y action (tipo de registro DNS).

```
        try:
            r =
requests.get(url='https://endpoint.apivoid.com/'+endpoint +
                '/v1/pay-as-you-
go/?key='+apivoid_key+'&host='+host+"&action="+action)
            return json.loads(r.content.decode())
        except:
            return {}
```

Construye URL de API con parámetro `action` para especificar tipo de registro DNS.

```
if __name__ == "__main__":
    apivoid_dns_lookup = ApivoidDnsLookup()
    json_data = apivoid_dns_lookup.apivoid_query('dnslookup',
'<<Host>>', 'dns-a')
    print(json.dumps(json_data, indent=4))
```

Bloque de ejecución que consulta registros A (IPv4) de un host.

Cómo Ejecutar el Script

```
python apivoid_dns_lookup.py
```

6.16 Script: apivoid_ssl_info.py

Introducción del Script

Este script consulta APIVoid para obtener información de certificados SSL/TLS de un dominio. Permite detectar certificados expirados, inválidos o sospechosos.

****Propósito**:** Analizar certificados SSL/TLS para detectar anomalías.

****Tipo**:** Herramienta de reconnaissance pasivo, SSL/TLS inspection.

Requisitos

- ****Librerías Python**:** `requests`
- ****API Key de APIVoid**:** Requerida

Explicación Línea por Línea

```
import requests
import json
```

Importa librerías.

```
apivoid_key = "<<API Key>>"
```

Define API key.

```
class ApivoidSslInfo:
    def apivoid_query(self, endpoint, host) -> json:
```

Define clase con método que acepta endpoint y host.

```
        try:
            r =
requests.get(url='https://endpoint.apivoid.com/'+endpoint +
                '/v1/pay-as-you-
go/?key='+apivoid_key+'&host='+host)
            return json.loads(r.content.decode())
        except:
            return {}
```

Construye URL de API, ejecuta request y parsea respuesta.

```
if __name__ == "__main__":
    apivoid_ssl_info = ApivoidSslInfo()
    json_data = apivoid_ssl_info.apivoid_query('sslinfo',
'<<Host>>')
    print(json.dumps(json_data, indent=4))
```

Bloque de ejecución que consulta información SSL de un host.

Cómo Ejecutar el Script

```
python apivoid_ssl_info.py
```

6.17 Script: crtsh_cert_info.py

Introducción del Script

Este script consulta crt.sh (Certificate Transparency logs) para obtener información de certificados SSL/TLS públicos asociados a un dominio. Permite identificar subdominios y analizar historial de certificados.

****Propósito**:** Enumerar subdominios y analizar certificados SSL/TLS.

****Tipo**:** Herramienta de reconnaissance pasivo, certificate analysis.

Requisitos

- ****Librerías Python**:** `pycrtsh`
- ****Instalación**:** `pip install pycrtsh`
- ****Acceso a Internet**:** Para consultar crt.sh

Dependencias Externas

- ****crt.sh**:** Base de datos pública de certificados. No requiere autenticación (disponible en <https://crt.sh/>).

Explicación Línea por Línea

```
from pycrtsh import Crtsh
import json
```

Importa clase `Crtsh` de la librería `pycrtsh`.

```
class CrtshCrtInfo:
    def cert_query(self, host):
```

Define clase con método que acepta host.

```
    crtsh = Crtsh()
```

Instancia objeto `Crtsh`.

```
    certs = crtsh.search(host)
```

Ejecuta búsqueda de certificados para el host especificado.

```
    print(json.dumps(certs, indent=4, default=str))
```

Imprime resultados en formato JSON. El parámetro `default=str` maneja objetos datetime que no son serializables por defecto.

```
if __name__ == "__main__":
    cert_info = CrtshCrtInfo()
    cert_info.cert_query('<<Host>>')
```

Bloque de ejecución que consulta certificados de un host.

Cómo Ejecutar el Script

```
python crtsh_cert_info.py
```

****Ejemplo con dominio personalizado:****

```
python -c "  
from pycrtsh import Crtsh  
import json  
crtsh = Crtsh()  
certs = crtsh.search('example.com')  
print(json.dumps(certs, indent=4, default=str))  
"
```

Resultado Esperado

Lista de certificados con información detallada:

```
[  
  {  
    "issuer_ca_id": 16418,  
    "issuer_name": "C=US, O=Let's Encrypt, CN=R3",  
    "common_name": "example.com",  
    "name_value": "example.com, www.example.com",  
    "id": 1234567890,  
    "entry_timestamp": "2023-01-15T10:30:00",  
    "not_before": "2023-01-15T00:00:00",  
    "not_after": "2024-01-15T00:00:00"  
  }  
]
```

Riesgos y Consideraciones Éticas

- ****Legalidad****: Consultar crt.sh es completamente legal y pasivo.
- ****Información Pública****: Los certificados son registros públicos.

Troubleshooting

- ****Resultados vacíos****: El dominio puede no tener certificados públicos.
- ****Timeout****: crt.sh puede estar lento. Reintentar después de unos segundos.

7. Implicancias de Ciberseguridad

El passive reconnaissance es la base de cualquier operación moderna de ciberseguridad. Las técnicas y herramientas presentadas en esta lección permiten:

****Evaluación de Postura de Seguridad****: Identificar qué información de infraestructura está expuesta públicamente y cuáles son los riesgos asociados.

****Descubrimiento de Activos****: Mapear completamente la superficie de ataque de una organización, incluyendo servidores, servicios, dominios y subdominios.

****Identificación de Vulnerabilidades****: Detectar servicios obsoletos, certificados expirados, configuraciones inseguras y software vulnerable.

****Threat Intelligence****: Correlacionar información de múltiples fuentes para construir perfiles de amenaza y detectar patrones de actividad maliciosa.

****Respuesta a Incidentes****: Utilizar técnicas de reconnaissance para investigar brechas de seguridad, rastrear infraestructura maliciosa y atribuir ataques.

8. Riesgos y Ética

Consideraciones Legales

Aunque el passive reconnaissance utiliza únicamente fuentes públicas, existen consideraciones legales importantes:

- ****Autorización****: Incluso las búsquedas pasivas deben realizarse dentro del alcance de una autorización formal (Rules of Engagement).
- ****Términos de Servicio****: Algunos servicios (Google, Shodan, Censys) tienen términos de servicio que prohíben ciertos tipos de automatización.
- ****Rate Limiting****: Ejecutar búsquedas excesivas puede violar políticas de uso aceptable.

Implicancias Éticas

- ****Responsabilidad de Divulgación****: Si se descubre información sensible o vulnerabilidades, debe reportarse responsablemente al propietario.
- ****Privacidad****: La geolocalización y otros datos pueden revelar información sensible sobre infraestructura.
- ****Propósito Legítimo****: Las herramientas deben utilizarse únicamente para propósitos autorizados de seguridad.

Impacto en Sistemas de Detección

Aunque el passive reconnaissance es pasivo por definición, las búsquedas automatizadas pueden:

- Generar tráfico anómalo que active alertas en WAF (Web Application Firewalls).
- Ser detectadas por análisis de logs de acceso.
- Violar políticas de rate limiting que pueden resultar en bloqueo de IP.

9. Buenas Prácticas

1. ****Documentación****: Mantener registro detallado de todas las búsquedas y hallazgos para auditoría y reproducibilidad.
2. ****Validación de Datos****: Cruzar información de múltiples fuentes para validar hallazgos.
3. ****Gestión de Credenciales****: Nunca hardcodear API keys en scripts. Utilizar variables de entorno o archivos de configuración seguros.

4. ****Cumplimiento de Límites****: Respetar rate limits de APIs y términos de servicio.
5. ****Privacidad de Datos****: Almacenar información sensible de manera segura y con acceso restringido.
6. ****Automatización Responsable****: Implementar delays entre búsquedas para evitar detección y bloqueo.

10. Troubleshooting

Problema	Causa Probable	Solución
"ModuleNotFoundError"	Librería no instalada	Ejecutar <code>`pip install [librería]`</code>
"Invalid API key"	API key incorrecta o expirada	Verificar credenciales en portal del servicio
"Quota exceeded"	Límite de consultas alcanzado	Esperar al período siguiente o actualizar plan
"Timeout"	Servidor lento o sin conectividad	Reintentar con delays entre consultas
"Empty results"	Dominio/IP no indexado	Verificar que dominio/IP sea válido y público
"Connection refused"	Firewall bloqueando acceso	Verificar conectividad a Internet y configuración de firewall
"Rate limited"	Demasiadas consultas en corto tiempo	Implementar delays entre requests
"FileNotFoundError"	Archivo no encontrado (ej. dorks.txt)	Verificar ruta del archivo y ajustarla en código

11. Conclusiones

Esta lección ha cubierto técnicas modernas de ****passive reconnaissance**** utilizando Python y APIs especializadas. Se han presentado 17 scripts que implementan:

- ****DNS Enumeration****: Mapear infraestructura de dominio mediante consultas DNS.
- ****Certificate Analysis****: Identificar subdominios y analizar certificados SSL/TLS.
- ****Threat Intelligence****: Evaluar reputación de IPs y dominios contra múltiples engines.
- ****OSINT Automation****: Automatizar búsquedas en Google y otras fuentes públicas.
- ****Attack Surface Discovery****: Mapear servicios expuestos en Internet.

Todas las herramientas y scripts han sido validados exhaustivamente para garantizar que los imports, librerías y métodos corresponden exactamente al código real entregado.

En la próxima lección, avanzaremos hacia técnicas de ****active reconnaissance****, que implican interacción directa con sistemas objetivo.